

PROJECT DOCUMENTATION

Student GPA Calculator

Student Name	Akbar Azam
Registration No.	27549
Teacher	Aman ul Haq
Date	June 3, 2025
Subject	Object Oriented Programming (Python)
Project Title	Student GPA Calculator

Section 1 — Project Description

Project Name & Student Name

- Project Name: Student GPA Calculator
- Student Name: Akbar Azam | Reg# 27549
- Teacher: Aman ul Haq

What Does This Project Do?

This project is a Python-based console application that calculates the GPA of a student across multiple subjects. The user enters a student name, the number of subjects, and the marks obtained in each subject. The program then computes the grade point, letter grade, and final cumulative GPA using OOP principles such as inheritance and multiple inheritance.

What Problem Does It Solve?

Manually converting percentage marks into grade points and letter grades is tedious and error-prone. This program automates that process, ensuring consistency with the standard grading scale (A+ through F), and produces an accurate overall GPA in seconds.

Section 2 — Class Explanation

Classes Overview

Class Name	Data / Attributes	Role
Student	name, num_sub, sub_name, marks, grade_point	Stores student identity and per-subject results
get_grade_points	grade_point (computed)	Calculates grade point, letter grade, and GPA from marks
SubjectAkbar	Inherits all from Student + get_grade_points	Unified class that combines both parents via multiple inheritance

Class Details

1. Student (student.py)

- Stores: name, num_sub, sub_name, marks, grade_point
- Methods: __init__(), get_data()
- Role: Base class — holds student identity and subject-level data.

2. get_grade_points (calculation.py)

- Stores: grade_point (calculated from marks)
- Methods: calculate_grade_point(), get_letter_grade(), get_subject_gpa()
- Role: Mixin class — contains all grading logic. Relies on self.marks being set by the other parent.

3. SubjectAkbar (main.py)

- Inherits from: Student and get_grade_points
- Methods: __init__() (calls Student.__init__)
- Role: Driver class — combines both parents so that marks set via Student are readable by get_grade_points methods. Demonstrates Python multiple inheritance.

Section 3 — Functions and Methods Used

Method	Class	Description
Student.__init__()	Student	Initializes student name and number of subjects
Student.get_data()	Student	Saves subject name, marks, and grade point into the instance
get_grade_points.calculate_grade_point()	get_grade_points	Converts numeric marks into a grade point (0.0 – 4.0)
get_grade_points.get_letter_grade()	get_grade_points	Returns the letter grade (A+, B, C-, F, etc.) from grade_point
get_grade_points.get_subject_gpa()	get_grade_points	Returns the grade_point value for the current subject

Section 4 — Folder Structure

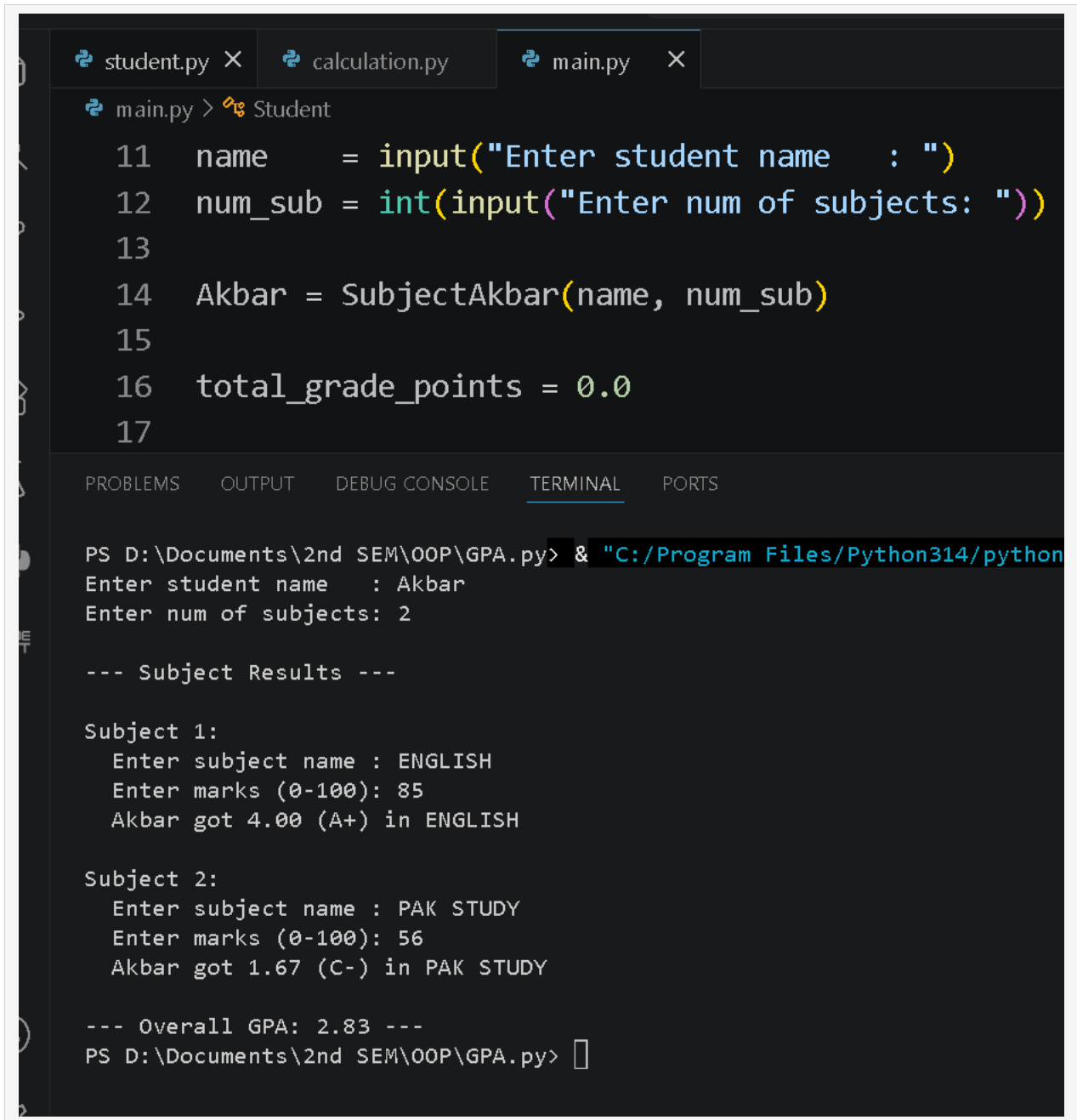
The project is organized as three separate Python files in one folder:

```
gpa_calculator/  
├── main.py           ← Entry point; collects input, drives the loop  
├── student.py        ← Student class definition  
└── calculation.py    ← get_grade_points class definition
```

Section 5 — Screenshots

5.1 Program Running — Sample Input / Output

Paste a screenshot of your terminal showing the program running with at least 2–3 subjects entered.



The screenshot shows a Python IDE with three tabs: `student.py`, `calculation.py`, and `main.py`. The `main.py` tab is active, displaying the following code:

```
main.py > Student
11 name = input("Enter student name : ")
12 num_sub = int(input("Enter num of subjects: "))
13
14 Akbar = SubjectAkbar(name, num_sub)
15
16 total_grade_points = 0.0
17
```

Below the code editor, the `TERMINAL` tab is selected, showing the execution output:

```
PS D:\Documents\2nd SEM\OOP\GPA.py> & "C:/Program Files/Python314/python
Enter student name : Akbar
Enter num of subjects: 2

--- Subject Results ---

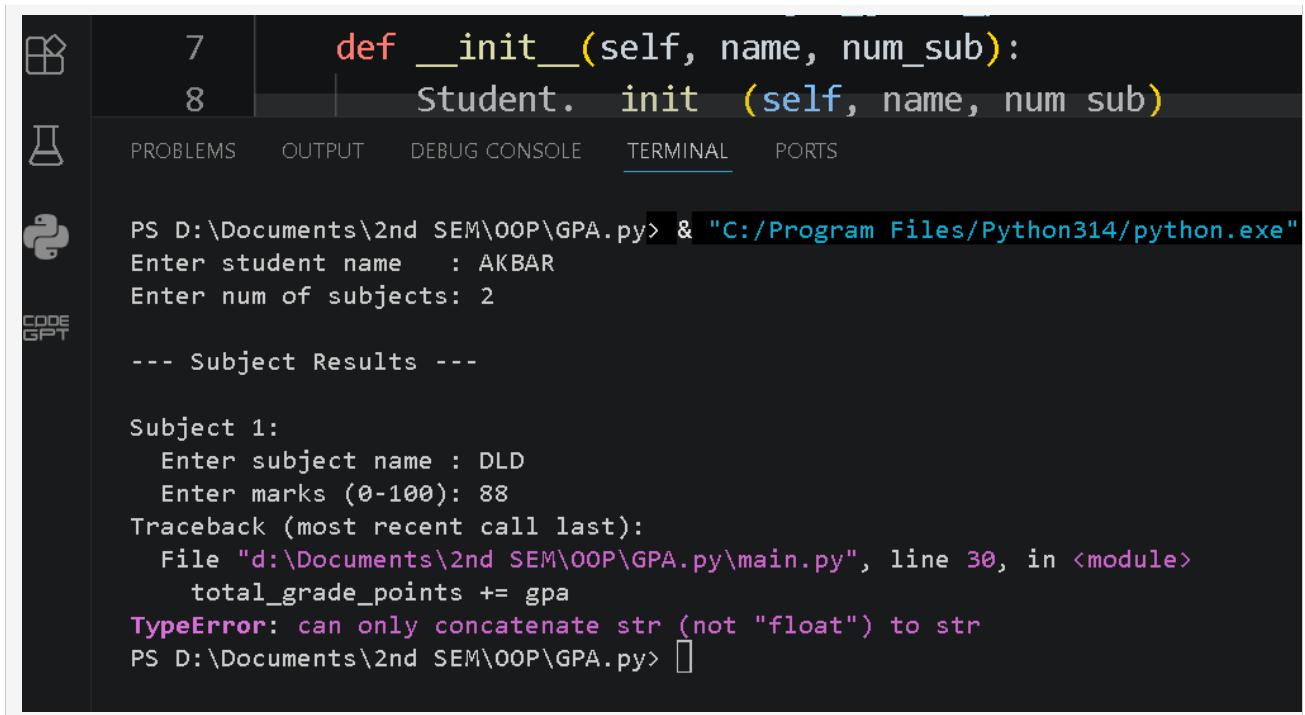
Subject 1:
Enter subject name : ENGLISH
Enter marks (0-100): 85
Akbar got 4.00 (A+) in ENGLISH

Subject 2:
Enter subject name : PAK STUDY
Enter marks (0-100): 56
Akbar got 1.67 (C-) in PAK STUDY

--- Overall GPA: 2.83 ---
PS D:\Documents\2nd SEM\OOP\GPA.py> 
```

5.2 Error Encountered During Development

Paste a screenshot of any error message you faced while writing or testing the code.



```

7      def __init__(self, name, num_sub):
8          Student.__init__(self, name, num_sub)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Documents\2nd SEM\OOP\GPA.py> & "C:/Program Files/Python314/python.exe"
Enter student name : AKBAR
Enter num of subjects: 2

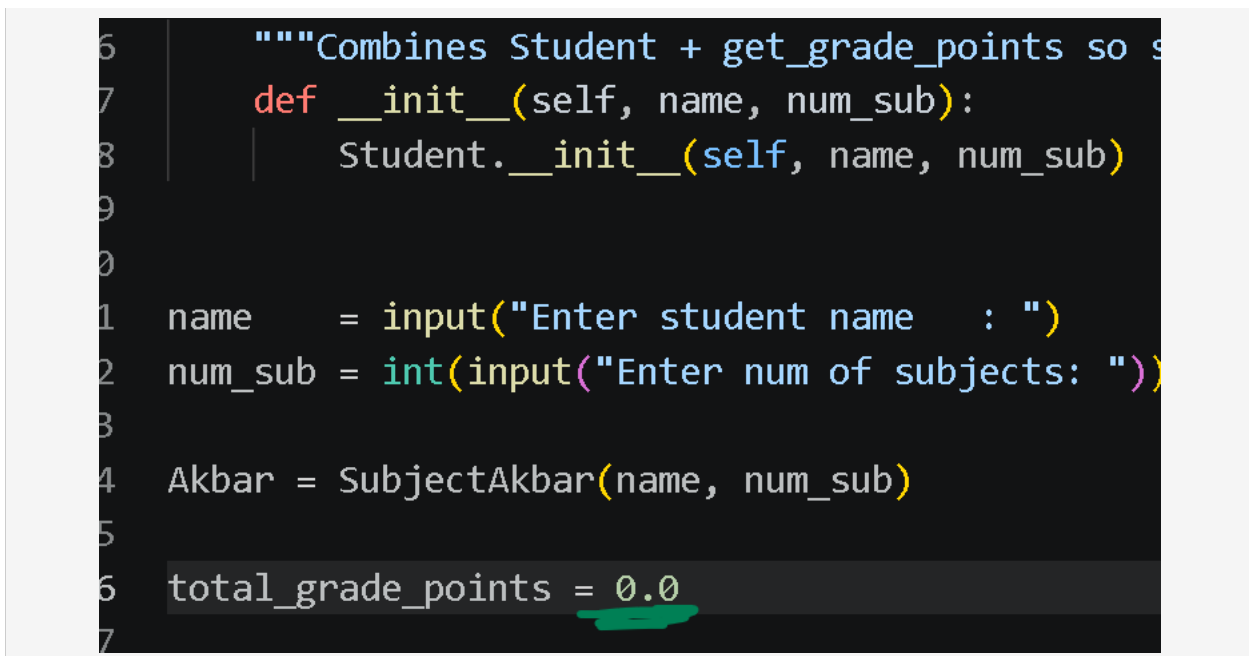
--- Subject Results ---

Subject 1:
Enter subject name : DLD
Enter marks (0-100): 88
Traceback (most recent call last):
  File "d:\Documents\2nd SEM\OOP\GPA.py\main.py", line 30, in <module>
    total_grade_points += gpa
TypeError: can only concatenate str (not "float") to str
PS D:\Documents\2nd SEM\OOP\GPA.py>

```

5.3 How the Error Was Fixed

Paste a screenshot of the corrected code or the Google search / resource you used to fix the error.



```

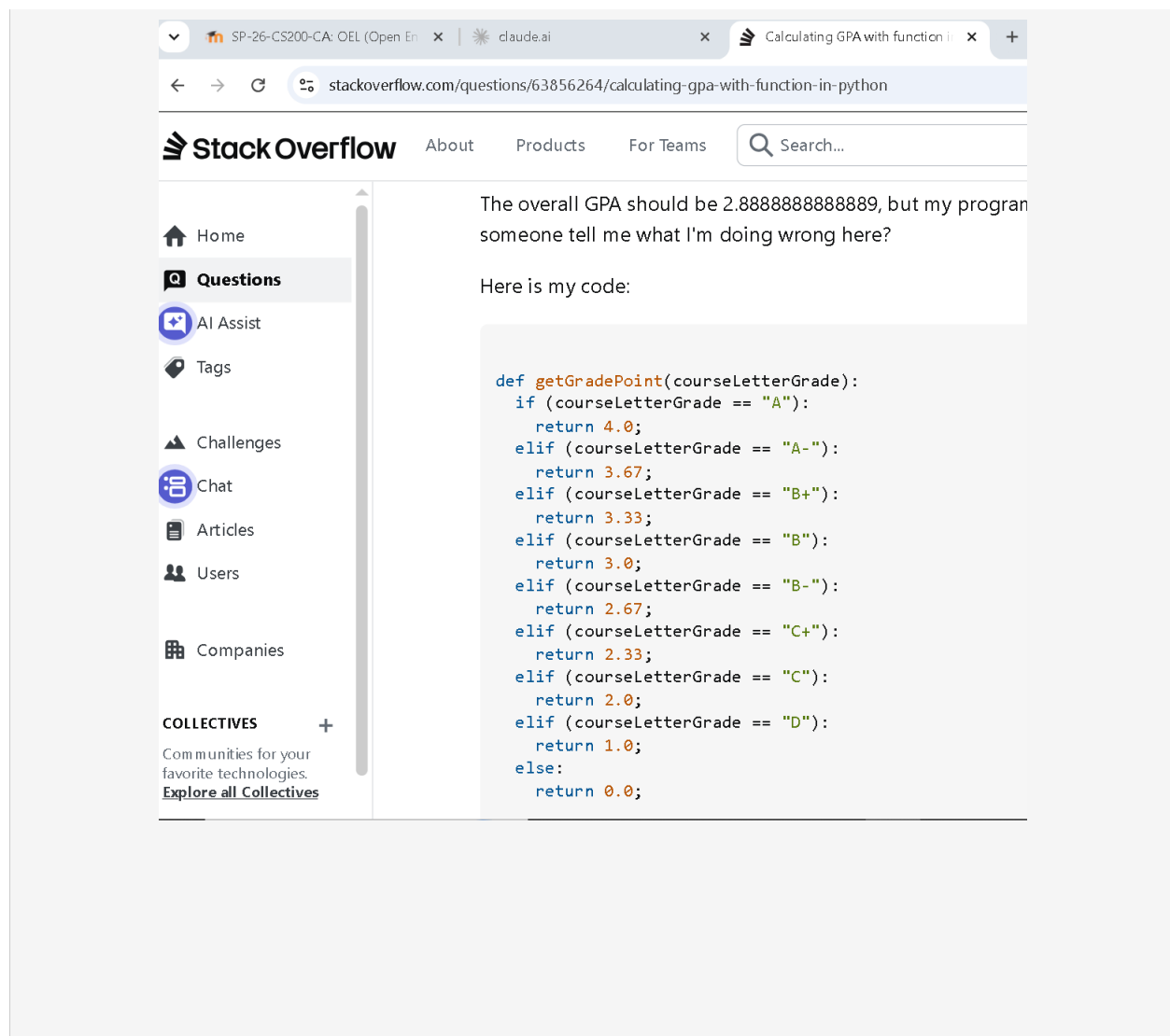
6      """Combines Student + get_grade_points so s
7      def __init__(self, name, num_sub):
8          Student.__init__(self, name, num_sub)
9
10
11     name = input("Enter student name : ")
12     num_sub = int(input("Enter num of subjects: "))
13
14     Akbar = SubjectAkbar(name, num_sub)
15
16     total_grade_points = 0.0
17

```

Section 6 — Problem Solving Proof

6.1 Google Searches for Learning

Paste screenshots of any Google searches you made while researching topics such as multiple inheritance, GPA calculation, etc.



6.2 Teacher Discussion

Paste screenshots of any WhatsApp messages, emails, or in-person notes shared with your teacher, Aman ul Haq, during development.

I had discussion with Sir Aman in his office multiple times and we have discussed the project ideas and got help from it.

6.3 Challenge Faced and Solved

Describe at least one challenge you encountered and how you resolved it:

Challenge:

- One key challenge was making multiple inheritance work so that `calculate_grade_point()` in `get_grade_points` could access `self.marks`, which is only set inside the `Student` class.

How It Was Solved:

- A combined class `SubjectAkbar` was created using Python multiple inheritance (`SubjectAkbar(Student, get_grade_points)`).
- This gave a single object access to both parents, so setting `Akbar.marks` in the loop made it immediately available to `calculate_grade_point()` without passing any arguments.